Project Progress Report: Privacy-Focused Research Agent

Mohammad Ammar, Divya Vinaybhai Patel, Samantha Susan John

1. Project Overview

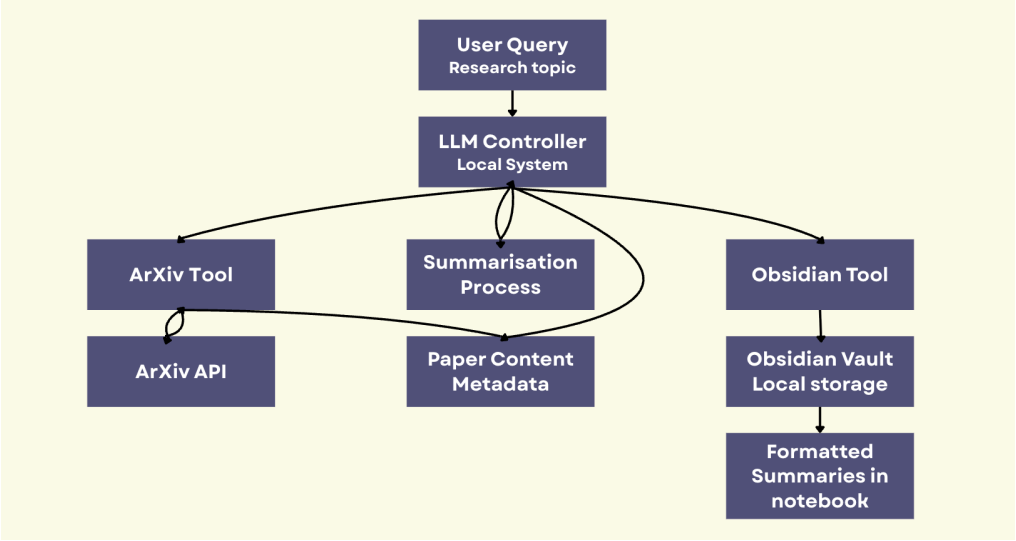
Our vision is to build a privacy-focused research assistant agent that utilizes the Model Context Protocol (MCP) to seamlessly connect our local LLM with powerful external research tools, including ArXiv, PDF parsers and Obsidian. Our primary goal is to empower users to autonomously perform deep literature reviews, extract actionable insights and generate connected, structured notes.

The core goal of Darwin is to provide an end to end research workflow that can search scholarly sources, retrieve papers, parse their content and convert insights into persistent research knowledge. The motivation is both practical and ethical. Practically, it reduces repetitive research overhead. Ethically, it supports data sovereignty by minimizing unnecessary exposure of sensitive research topics and unpublished ideas.

Darwin treats privacy as a design requirement, not a side feature. Papers are downloaded locally, extraction is done locally and the architecture is built so users can choose local model inference or API based inference depending on their setup and risk tolerance.

Darwin follows a modular MCP based architecture with a central orchestrator and specialized tool servers. The orchestrator is responsible for session control, conversation state, model interaction and tool execution flow. It receives user requests, queries the model for reasoning and action planning, executes tool calls through MCP and then feeds tool outputs back to the model until a complete answer is produced.

2. Architecture



Darwin is built as a modular research agent where a central orchestration layer manages language model reasoning and delegates specialized tasks to MCP tools. The architecture follows a clear separation of concerns. The agent core handles conversation state and decision flow, the model client layer handles inference and the MCP server layer handles external actions such as paper search, paper download and document parsing. This separation has been useful during development because each layer can be tested and improved without rewriting the rest of the system.

At runtime, the user interacts through a terminal interface. The agent loads configuration, selects a provider, then connects to available MCP servers using the local configuration file. After the model is initialized, the agent discovers all available tools from connected MCP servers and enters a tool execution loop. In this loop, the model can request one or more tools, the agent executes them, appends the results to message history and returns control to the model for synthesis. This loop continues until the model returns a final response with no further tool requests.

- Foundation behavior is already real and runnable, not just conceptual. The startup flow loads provider configuration, initializes the selected model client, connects to MCP servers, discovers tool definitions and then enters a multi turn execution loop. That means the core architecture already supports end to end execution from user query to tool result to model synthesis.
- The MCP integration is modular and extensible. Each configured server is opened as its own session, tool schemas are normalized into function calling format and tools are invoked by name through server side discovery.
- Multi provider model support is implemented as interchangeable clients. We expose a compatible chat interface, which is why the orchestrator can switch between local and API based inference through config alone. Architecturally, this reduces coupling and improves deployment flexibility across machines and environments.
- ArXiv operations already cover the full paper lifecycle. The server performs discovery, retrieval, local persistence and text extraction. The read path includes bounded extraction through max_chars, which protects the control loop from oversized context payloads while still preserving meaningful content for summarization.

- PDF support exists in two layers. First, native extraction is integrated in read_paper. Second, dedicated parsing tools provide a separate service boundary for section level handling. This layered design is useful to mention because it gives you a practical path from baseline extraction to richer document intelligence without disrupting the stable ArXiv flow.

3. Current Implementation Status/ Method

Phase 1: Foundation Setup

- **Local LLM & MCP Host**: **In Progress**. We have implemented a custom research agent orchestrator in `src/agent/main.py`. This solidifies our foundation by directly integrating our standalone `OllamaClient` with our `MCPClient`.

  The current implementation initializes runtime configuration from a single agent configuration file, supports provider selection at startup and then constructs the model client and MCP client as independent modules. This separation allows us to switch model backends without changing orchestration logic. At runtime, the orchestrator establishes MCP sessions first, discovers available tools from connected servers and passes tool schemas to the language model for autonomous function selection. This confirms that our host layer is already capable of real tool mediated execution rather than a hard coded command flow. We also validated startup behavior under missing configuration conditions through fallback defaults, which improves reproducibility during local testing.

- **ArXiv MCP Server**: **Implemented**. We have successfully built our custom ArXiv server using `FastMCP` in `src/arxiv_server/server.py`.

- We have exposed critical tools: `search_papers`, `download_paper`, `list_papers` and `read_paper`.
- We have successfully integrated **PDF Parsing** into our `read_paper` tool, leveraging `pymupdf` to natively extract text from our downloaded papers.

The ArXiv server now supports the full paper lifecycle from discovery to local content access. The search path uses arxiv client queries with bounded result counts and returns structured metadata fields that the agent can directly consume for follow up decisions. The download path stores files in a deterministic local repository and checks for existing files before fetching, which reduces redundant network requests and supports cache reuse across sessions. The read path opens local PDFs, extracts page text incrementally and applies a configurable character cap to prevent oversized context payloads during model inference. This design has been tested through standalone scripts and integrated runs, confirming that search, retrieval, storage and extraction are functionally connected.

```
[03/25/26 23:44:04] INFO     Starting MCP server 'PDFParser' with transport   transport.py:183
                             'stdio'
Connected to pdf_tools
Discovering tools...
Found 8 tools.
  - search_papers
  - download_paper
  - list_papers
  - read_paper
  - confirm_download
  - log_research_action
  - extract_pdf_sections
  - extract_figures

Agent ready. Type 'exit' to quit.
```

- **Obsidian MCP Server**: **Pending**. While we have defined the conceptual structure for Obsidian integration in our Architecture docs, we have not yet established the configuration or connection logic within our active source tree.

  At this stage, the architecture already allocates a role for persistent knowledge management, but the live runtime configuration does not yet register an Obsidian MCP endpoint. No production path currently writes synthesized outputs into vault notes, tags, or linked references. This means Darwin can complete retrieval and analysis tasks but does not yet persist them into a graph based knowledge layer. The pending work is implementation oriented rather than conceptual: server installation, environment configuration, MCP registration and validation of note creation and retrieval operations through the same tool loop used by ArXiv and PDF services.

Phase 2: Agentic Workflow

- **Research Assistant Control Loop**: **Implemented**. In our orchestrator (`src/agent/main.py`), we have established a robust, conversational chat loop. This

allows our agent to connect to the local LLM, dynamically discover the available MCP tools and execute tool calls autonomously based on its reasoning.

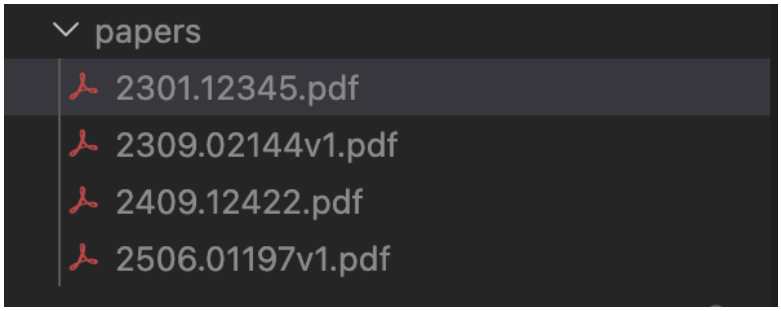
The implemented loop operates as a multi turn execution cycle where model responses are parsed for tool calls, tool calls are executed against MCP servers, results are appended to message history and the model is reinvoked until it returns a final non tool response. This creates a closed reasoning and action loop suitable for compound tasks such as finding papers, selecting candidates, downloading chosen entries and summarizing extracted content in one conversation. The loop also preserves role structured history, including tool call identifiers where required by API compatible endpoints, which is critical for stable tool response grounding. From a method perspective, this is the central mechanism that transforms Darwin from a static chatbot into an agentic research workflow engine.

```
You: Find exactly 3 highly relevant recent papers on mechanistic interpretability in mac
hine learning using ArXiv tool results only, and return title, ArXiv ID, date, and one s
entence on why each is mechanistic interpretability.
Sending request to llama-3.1-8b-instant...
Executing tool: search_papers...
Args: {'max_results': 3, 'query': 'mechanistic interpretability in machine learning'}
Calling search_papers on arxiv...
2026-03-25 23:49:31.477 | INFO     | __main__:search_papers:32 - Searching for mechanist
ic interpretability in machine learning...
2026-03-25 23:49:31.478 | INFO     | __main__:search_papers:41 - Search query: 'mechanis
tic interpretability in machine learning'
2026-03-25 23:49:31.722 | INFO     | __main__:search_papers:44 - Found paper: Changing D
ata Sources in the Age of Machine Learning for Official Statistics (2306.04338v1)
2026-03-25 23:49:31.723 | INFO     | __main__:search_papers:44 - Found paper: Model-Agno
stic Interpretability of Machine Learning (1606.05386v1)
2026-03-25 23:49:31.723 | INFO     | __main__:search_papers:44 - Found paper: Why Interp
retability in Machine Learning? An Answer Using Distributed Detection and Data Fusion Th
eory (1806.09710v1)
2026-03-25 23:49:31.723 | INFO     | __main__:search_papers:52 - Returning 3 results
Tool result: [{"id":"2306.04338v1","title":"Changing Data Sources in the Age of Machine
Learning for Official Statistics","published":"2023-06-07","summary":"Data science has b
ecome increasingly essential for the ...
Sending request to llama-3.1-8b-instant...
```

OUTPUT:

```
1. **Model-Agnostic Interpretability of Machine Learning** (1606.05386v1, 2016-06-16) -
This paper proposes a model-agnostic approach to explain machine learning predictions, e
nabling flexibility in the choice of models, explanations, and representations.
2. **Changing Data Sources in the Age of Machine Learning for Official Statistics** (230
6.04338v1, 2023-06-07) - This paper highlights the risks and liabilities associated with
 changing data sources in machine learning for official statistics, emphasizing the impo
rtance of robustness and data sourcing techniques to maintain integrity and reliability.
3. **Why Interpretability in Machine Learning? An Answer Using Distributed Detection and
 Data Fusion Theory** (1806.09710v1, 2018-06-25) - This paper argues that human interpre
tability of machine learning models is important, and characterizes the performance of a
 human-machine learning system using the theory of distributed detection, demonstrating
the superiority of interpretable classifiers over black box classifiers.
```

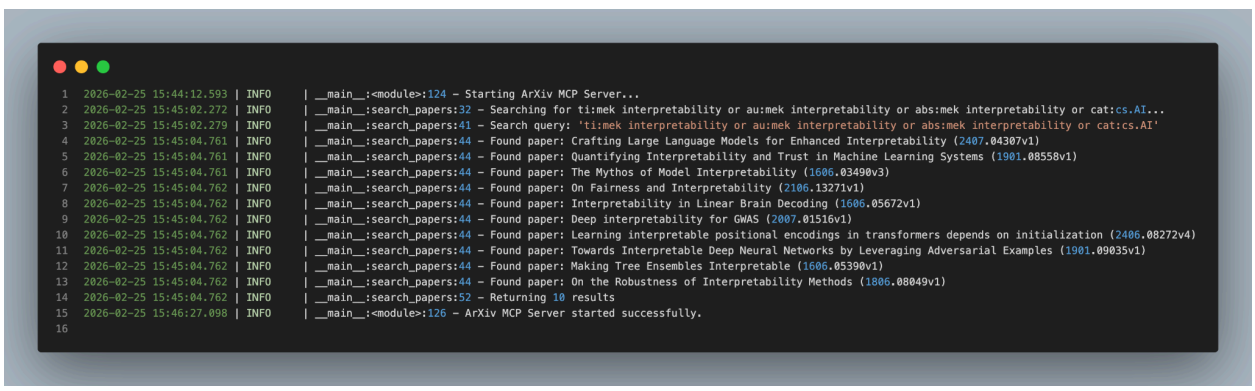
PAPERS AUTO-DOWNLOAD:



Phase 3: Production Features

- **Logging & Evaluation**: **Implemented**. We have introduced robust tracking for all agent actions via the `log_research_action` tool. This ensures our data is persisted correctly to our local SQLite database (`research_log.db`), giving us full visibility into the agent's behavior.

  The logging layer currently captures timestamped action records with operation type, paper identifier and result payload, enabling traceability across complete runs. This gives us an auditable execution trail that can be used for debugging, error diagnosis and post run quality evaluation.



```
2026-02-25 15:44:12.593 | INFO     | __main__:<module>:124 - Starting ArXiv MCP Server...
2026-02-25 15:45:02.272 | INFO     | __main__:search_papers:32 - Searching for ti:mek interpretability or au:mek interpretability or abs:mek interpretability or cat:cs.AI...
2026-02-25 15:45:02.279 | INFO     | __main__:search_papers:41 - Search query: 'ti:mek interpretability or au:mek interpretability or abs:mek interpretability or cat:cs.AI'
2026-02-25 15:45:04.761 | INFO     | __main__:search_papers:44 - Found paper: Crafting Large Language Models for Enhanced Interpretability (2407.04307v1)
2026-02-25 15:45:04.761 | INFO     | __main__:search_papers:44 - Found paper: Quantifying Interpretability and Trust in Machine Learning Systems (1901.08558v1)
2026-02-25 15:45:04.761 | INFO     | __main__:search_papers:44 - Found paper: The Mythos of Model Interpretability (1606.03490v3)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: On Fairness and Interpretability (2106.13271v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: Interpretability in Linear Brain Decoding (1606.05672v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: Deep interpretability for GWAS (2007.01516v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: Learning interpretable positional encodings in transformers depends on initialization (2406.08272v4)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: Towards Interpretable Deep Neural Networks by Leveraging Adversarial Examples (1901.09035v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: Making Tree Ensembles Interpretable (1606.05390v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:44 - Found paper: On the Robustness of Interpretability Methods (1806.08049v1)
2026-02-25 15:45:04.762 | INFO     | __main__:search_papers:52 - Returning 10 results
2026-02-25 15:46:27.098 | INFO     | __main__:<module>:126 - ArXiv MCP Server started successfully.
```

- **Human-in-the-Loop**: **Partial**. We have laid the groundwork via the `confirm_download` tool in our ArXiv server. Currently, this serves as a placeholder stub that logs the request and returns `True`, ensuring steady workflow execution.

  From a systems method perspective, this is an intentional scaffold that reserves a policy checkpoint in the execution path without blocking integration testing. The existing interface makes it straightforward to upgrade into a true interactive approval gate because call signatures and invocation points are already defined. Once interactive prompting is added, the same control point will enforce user confirmation before high cost or high volume retrieval actions, improving operational safety and user oversight while preserving current orchestration flow.

- **Caching**: **Implemented**. We have implemented local caching by saving downloaded files into a dedicated `papers/` repository, ensuring we never needlessly re-download files. Rate limiting for external APIs is slated for a future update.

  The cache behavior is file based and deterministic. Before initiating a download, the server checks local storage for an existing matching paper identifier and returns the local path when available. This reduces latency, lowers external request volume and improves consistency of repeated experiments because the same source artifact is reused.

- **Error Handling**: **Implemented**. We have established foundational error guards (`try/except` blocks) to catch common PDF extraction faults and prevent catastrophic server crashes.

  Current safeguards are concentrated around high risk operations such as file access, PDF parsing and network dependent retrieval paths. Failures are converted into controlled error responses, allowing the agent loop to continue and report actionable feedback instead of terminating unexpectedly. This improves robustness during heterogeneous document processing where malformed files, missing resources, or parser edge cases are common.

3. Next Steps & Recommendations

1. **Integrate Obsidian Server**: Our immediate next step is to install and configure the `obsidian-mcp-server`. This will open the pathway for our agent to save the intelligence it mines directly into the user's Obsidian knowledge vault.

   This step moves Darwin from a retrieval and analysis tool into a long term research memory system. Once the server is integrated, the agent will be able to create structured notes for each paper, including title, authors, abstract summary, methods, findings and direct links to local PDF artifacts. It will also allow automatic tagging by topic, method family and publication year, which improves later search and thematic grouping.

2. **Complete the Human-in-the-Loop Feature**: We need to update the `confirm_download` tool so that it accurately prompts for human input on the command-line before downloading papers. This ensures responsible content retrieval.

   The implementation should introduce a true interactive decision gate in the execution path so users can approve or reject candidate papers before network download occurs. The prompt should display key context such as title, paper ID, publication date and a short abstract preview to support informed decisions quickly. This mechanism is especially important when users request broad queries that return large result sets, where uncontrolled auto download can waste time and storage. Adding this feature will improve operational safety, user trust and reproducibility, because the selected paper set becomes an explicit human validated subset rather than a purely model chosen sample.

3. **Refine the Agent's Prompt Context**: We should finalize and pass the "Weekly Research Digest" system prompt from our design docs into `config/agent_config.json`, locking the agent deeply into its dedicated literature-review persona.

A finalized digest prompt will standardize behavior across sessions and reduce variation in output quality. The prompt should define a stable report schema for each run, including research objective, selected papers, per paper contribution summary, methodology comparison, limitations, and actionable follow up questions.

Darwin has successfully established a working, privacy focused research pipeline that already delivers real end to end value through MCP based paper search, retrieval, and analysis, and with the planned Obsidian integration and human approval enhancements, it is well positioned to become a robust, production ready assistant for trustworthy and repeatable literature review workflows.